Practice Guide 2: From Words to Meaning (Word Vectors, Small Language Models, llama.cpp)

Aprendizagem Aplicada à Segurança

Mário Antunes

October 2, 2026

Objective

Climb the second half of the spam ladder. You will train **Word2Vec** and **fastText** vectors on our messages and compare them with **pre-trained** ones, use the **sentence embeddings** of a small language model (MiniLM), and then run **EmbeddingGemma 2**, both in your own **llama.cpp servers** (Docker, GPU or CPU), as feature extractors. You will find out **when** each one is the right choice, and how much a single F1 number can be trusted.

- **Notebook to complete:** `guide_02_02.ipynb`. Cells marked `TODO` are yours; `check` cells validate your work.
- **Time:** about 2 hours (suggested: A 5 min, B 20, C 20, D 15, E 40, F 15).
- **Prerequisite:** Guide 1 and the lecture of class 3.
- **Requirements:** `make venv` at the root of the repository (no PyTorch, no TensorFlow). For parts D and E you need **Docker** and the model files: from the repository root run `make llama-models` (downloads about 2.8 GB: EmbeddingGemma 2 176 MB, MiniLM 21 MB and the small chat model Gemma 4 E2B 2.6 GB; checksums verified), then `make llama-up` (it detects your GPU and picks the matching image; `PROFILE=cpu` forces the CPU) and `make llama-check`. One **llama-swap** endpoint on port 8080 serves every model; the `model` field of the request picks it and the first request after a swap waits while the GGUF loads. Details and troubleshooting: `notebooks/01-spam/llama/README.md`.
- Set `AAS_W2V=skip` to skip the 1.7 GB Google News download (part C then compares two sources instead of three).

Background

Word2Vec. One vector per word, learnt from the words around it (no labels). *Skip-gram* predicts the context from the word; negative sampling pushes the score of a true pair up and the score of k random words down. A message is the **mean** of the vectors of its words; a standardised logistic regression classifies it.

fastText. A word vector is the sum of the vectors of its character n-grams, so unseen and misspelt words (`fr33`, `freee`) still get a vector. A *pre-trained* model is only as good as its coverage of your vocabulary: the compressed Common Crawl model we use keeps 20,000 words and prunes many n-grams.

Sentence embeddings. A frozen Transformer maps the whole message to one vector; only a small classifier is trained (*linear probe*). Standardise each dimension first (embeddings are not centred). MiniLM (22 M parameters) and EmbeddingGemma 2 (270 M) both run in llama.cpp servers.

EmbeddingGemma 2. 270 M parameters (a text encoder from the Gemma family), 768-d vectors, *Matryoshka* training (the first d values are a valid smaller embedding), a task prefix (`title: none | text:`) in front of each text. It is served by **llama.cpp** from a **GGUF** file (here quantised to 4 bits, `UD-Q4_K_XL`: the weights are stored as 4-bit integers with a scale per block, so the file is 176 MB instead of 558 MB at 16 bits); the notebook talks to it over HTTP (`/v1/embeddings`, OpenAI-compatible) and receives L2-normalised vectors.

Intervals. The test set has 1,024 messages (about 130 spam): a difference of 0.005 in F1 is noise. A *bootstrap interval* resamples the test set with replacement and recomputes the F1 many times; its 2.5% and 97.5% percentiles bracket the score.

Tasks

Part	What you do	Check
A	Data, helpers and the TF-IDF reference (given)	
B	Train **Word2Vec** (skip-gram); implement the **mean message vector**	`message_vector` OK, Questions B1, B2
C	Implement the **OOV rate**; compare in-domain and pre-trained vectors; train **fastText** with n-grams and test **leet-speak**	table, Questions C1 to C3
D	Embed with **MiniLM** (`model minilm`); linear probe	F1 close to 0.95, Question D1

Part	What you do	Check
E	Raw HTTP request to the **llama.cpp** stack (`model embgemma`); **Matryoshka** truncation; **bootstrap interval** for the prefix	shapes and cosines, Questions E1 to E3
F	Few labels and the summary table	Questions F1 to F3

Tips. If part E cannot reach the server, run `make llama-check` and read `docker compose logs`; the notebook prints the hint. The first embedding of the 5,119 messages takes a few minutes and is cached in `notebooks/01-spam/.cache/`.

Questions

1. **(B1)** Look at the nearest neighbours of `free`, `prize`, `call` and `home`. What kind of words do you get, and why do they differ from a dictionary?
2. **(B2)** The corpus has about 65,000 tokens. Which words get unreliable vectors, and why?
3. **(C1)** Print ten frequent spam tokens that Google News has no vector for. What do they have in common, and why does it matter for a spam filter?
4. **(C2)** Which source wins on this corpus? What would change if the corpus were product reviews?
5. **(C3)** Report the recall of TF-IDF and of the in-domain fastText on the leet-speak spam (and on the clean spam). Explain the difference. Which disguise would hurt fastText more (think of `f.r.e.e`)?
6. **(D1)** Why do we standardise the embeddings, and why fit the scaler on the training set only?
7. **(E1)** Why is the cosine of the first pair high although the messages share almost no word? What would TF-IDF give?
8. **(E2)** What is the smallest number of dimensions whose F1 is within 0.01 of the full vector, and how many bytes per million messages does it save?
9. **(E3)** Do the bootstrap intervals of the two prefixes overlap? What do you conclude for *classification*, and why could the answer be different for retrieval?
10. **(F1)** Fill in the table below from your results.
11. **(F2)** In which range of labelled examples do the embeddings win, and why do they need fewer labels?
12. **(F3)** Choose a model for a mail gateway that handles 10,000 messages per second and receives few new labels. Justify with cost per message, accuracy and robustness.

Model	F1 (all labels)	F1 (50 labels)	cost per message
TF-IDF + logistic regression			
Word2Vec in-domain + LR			
MiniLM + LR			
EmbeddingGemma 2 (llama.cpp) + LR			

Optional challenges

- Restart the server with the smaller `Q4_K_M` file (`download_model.py --quant Q4_K_M`, `EMBED_GGUF` in `.env`) and repeat part E. What do you gain and lose?
- Replace the mean by a **TF-IDF-weighted mean** of the word vectors. Does it help the static vectors?
- Repeat part F with **leave-one-family-out**: `spam_families` gives six scam families; train without one, test on it (notebook 07, section 6).
- Concatenate the MiniLM and EmbeddingGemma vectors and train one probe. Does it beat either alone, and what does it cost per message?

Reference solution

The solved notebook is `solution/guide_02_02.ipynb`.